17 — The single-pool operational model (v16.0)

> **Citations to files that are not in this tree.** An audit found 39 paths cited across 19 documents that no longer exist — the fourteen-pool model, the v1/v2 generators and the drawing scripts that went with them, and the Sprint 0 Node scaffold. Nothing was lost: all of it is in git history, and every citation is listed with what replaced it in 18-cited-files-no-longer-present.md.

This is the shipped operational model. `bpm/model/UFCEP6-0-3_Hospital_Patient_Pathway_v16.bpmn`

---

1. What changed, and why

v14.0 drew the hospital as **fourteen pools**. Nine of them were the hospital's own desks — Medical Secretaries, Outpatient Bookings, Treatment Bookings, Consultants, the Clinical Nurse Specialist Team, Call Handling, the Patient Pathway Coordinators, Administrative Management and Finance — each as its own deployable process, and the desks reached each other by **message flow**.

That is the wrong construct. A pool is an independent participant; a message flow is an agreement between two of them. Between the referral desk and the consultant there is no agreement, only a hand-off, and BPMN already has a construct for a hand-off inside one organisation: a **sequence flow**, with the desks separated by **lanes**. The v14.0 drawing also had no single process that could be pointed at and called "the pathway" — the patient journey existed only as a chain of messages between nine unrelated process definitions, and no one could follow it without reading all nine.

v16.0 is one pool, the hospital, with the nine desks as lanes:

Responsibility lane	Team lanes inside it
Clinical responsibility	Consultants, Clinical Nurse Specialist Team
Administrative responsibility	Medical Secretaries, Outpatient Bookings Team, Treatment and Chemotherapy Bookings Team, Call Handling Team, Patient Pathway Coordinators, Administrative Management Team
Financial responsibility	Finance Team

Message flows survive only where an independent participant really is on the other end: the referring organisation, the patient, and the four outside services the hospital buys capacity, distribution or payment from. They are drawn as black-box participants in a row above the pool.

The shape now matches the strategic model in `UFCEP4-0-3`, which resolves the same seven participants and the same three responsibility lanes.

---

2. What the model contains

	v14.0	v16.0
Participants	14 pools	**7** — 1 hospital pool + 6 outside participants
Executable process definitions	9	**1**
Lanes	19 (flat)	**3 responsibility lanes holding 9 team lanes**
Flow nodes	355	**166** (150 flow nodes + 16 boundary events)
Sequence flows	312	**180**
Message flows	20	**15**
User tasks	39	**38**, every one with a Camunda Form and a candidate group
Service tasks	39	**48**
Gateways	—	**32** (29 exclusive, 2 inclusive, 1 event-based)
Boundary events	—	**16** (11 error, 3 non-interrupting timer, 1 non-interrupting message, 1 compensation)
Start / end events	—	**3 start** (1 none, 1 message, 1 timer) / **22 end**

The end events are deliberate. Every terminal outcome of the pathway has its own named end — referral not accepted, information never received, patient declines, request not authorised, provisional series released, treatment change refused, refund with the provider, payment unresolved, pathway complete — because a model that funnels six different outcomes into one circle tells the reader that the outcomes are the same when they are not.

The pathway a reader can follow without leaving the diagram

```
referral in -> pack checked (and chased if incomplete)
  -> clinically reviewed -> accepted or not
  -> new patient appointment booked, letter sent, telephone contact recorded
  -> attended -> consent recorded -> treatment booking request authorised
  -> clinic letter drafted, approved, processed, dispatched
  -> funding route established -> charge, payment, or an urgent-treatment authorisation
  -> provisional treatment series planned against external capacity
  -> cycle scheduled -> clinically reviewed -> continued, delayed or changed
  -> follow-up booked -> outcome recorded -> rebook, settle the account, or close
```

Running alongside it, in the same process: the enquiry path (administrative, financial or clinical; urgent or routine), and the weekly correspondence sweep with its one-month and three-month escalations.

3. Evidence

Structural — `tools/validate_model.js`, i.e. what the Modeler's status bar shows

```
$ NODE_MODULES_DIR=... node tools/validate_model.js \
      model/UFCEP6-0-3_Hospital_Patient_Pathway_v16.bpmn

parse warnings      : 0
unresolved refs     : 0
participants        : 7
processes           : 1 | executable: 1
dangling seq refs   : 0
user tasks no form  : 0
user tasks no group : 0
service tasks no job type: 0
RESULT: clean — Modeler would report no problems
```

`tools/analyse_bpmn.py` adds: 167 flow nodes, 181 sequence flows, 15 message flows, 0 elements without diagram interchange, 0 user tasks without a form or a role.

Drawing quality — `tools/analyse_layout.py`

	v14.0	v16.0
Overlapping parallel sequence-flow segments	0	**0**
Overlapping parallel message-flow segments	0	**0**
Diagonal segments	0	**0**
Lines drawn through an unrelated shape	5	**0**
Sequence-flow crossings	54	105
Mean bends per flow	0.7	1.5 (max 4)

The crossing count is higher, and that is the honest cost of the change: seventeen pools stacked down a page let each pool keep its lines to itself, whereas one pool 19 rows deep has to route every hand-off across the lanes it passes. What is bought for it is that nothing is drawn through a shape and nothing is drawn on top of anything else — v14.0 had five lines through shapes it was not connected to.

Reconciliation with the rest of the release

Check	Result
`.form` files bound in the model	**36 of 36**, no unbound file, no missing file
Worker job types required by the model	**36**, and `@JobWorker` classes declare **36** — no orphan on either side
User tasks without a form or a candidate group	0

The four job types the first draft of this model stopped using — `correspondence.prepare-dispatch`, `enquiry.record-contact-attempt`, `finance.prepare-refund`, `pathway.find-overdue-clinic-letters` — were put back as activities rather than left as dead code, so every worker in `bpm/workers` still has a job to do.

The drawing

File	What it is
`diagram/hospital-patient-pathway-v16.svg`	the vector original, 17,715 × 3,990
`diagram/hospital-patient-pathway-v16.png`	the same drawing rasterised
`diagram/hospital-patient-pathway-v16-overview.png`	whole drawing, scaled to 2,400 px
`diagram/hospital-patient-pathway-v16.pdf`	full drawing on one page, vector
`diagram/hospital-patient-pathway-v16-sections.pdf`	**seven captioned stages**, cut from the full render

Regenerate with

```
python3 tools/build_v3.py                                        # the .bpmn
python3 tools/render_v16.py model/UFCEP6-0-3_Hospital_Patient_Pathway_v16.bpmn \
        diagram/hospital-patient-pathway-v16                     # svg, png, pdf
python3 tools/render_sections_v16.py diagram/hospital-patient-pathway-v16.png \
        diagram/hospital-patient-pathway-v16-sections.pdf        # the sections
```

---

4. Decisions a reader might question

The pathway opens at a none start event, not a message start event. A referral arriving from an outside organisation is the textbook case for a message start event, and v14.0 used one. It is not used here: the process begins at a plain start circle, and a case is opened by `CreateProcessInstance` carrying the referral variables. Two reasons. First, the

whole hospital is one pool, and a start event that only ever fires from inside the collaboration reads as a second entry point rather than as the pathway's beginning. Second, the referral is the *reason a case exists*, not an event the running pathway reacts to, so making it the trigger to create the instance states that more plainly than a subscription does.

The referring organisation is not dropped from the collaboration by this. Its hand-off is now drawn against the reply it sends to the missing-information request, `SEC_Catch_InfoReceived` — a hand-off that was not drawn at all before, so the collaboration is one arrow better off than it was. The other two referrer exchanges (the request for information and the decision back) are unchanged.

Camunda 8 permits one none start event together with message start events, so this does not conflict with the enquiry and weekly-review entry points, which stay as they were: `Start_Enquiry` is still a message start event and `PCW_Start_WeeklyReview` is still a timer start event. `referral.received` is therefore no longer declared as a message, and publishing it starts nothing.

Three waits for an outside message were replaced by things a person does. v16.0 originally parked the pathway at three message catch events — `appointment.attended`, `appointment.outcome` and the reply to a request for missing information — each of which needed something outside the engine to publish a message before the case could move. That is defensible modelling and it is what the v14.0 collaboration did, but it has a practical cost: **Tasklist cannot publish a message**, so a case could not be driven to its end from the user interface alone. Every demonstration needed a script alongside it.

Two of the three are now user tasks, and the third is gone because it was redundant:

Was	Now	Why
`OUT_Catch_PatientAttended` (message `appointment.attended`)	**`OUT_Task_RecordAttendance`**, form `record-attendance`	The bookings team marking the patient in is the same business step, and it is what the next task already assumes.
`OUT_Catch_AppointmentOutcome` (message `appointment.outcome`)	**removed**; `OUT_Auto_SendFollowUpLetter` runs straight into `OUT_Task_DecideNextAction`	The very next task's form already captures `appointmentOutcome` — the value `booking.record-appointment-outcome` validates two steps later. The wait asked a person to enter an outcome they had just been asked for.
`SEC_EGW_InfoRequest` (message `referral.info-received`)	**unchanged**	It is the missing-information loop, and a loop that waits for the reply is the point of it. A pack that passes the checklist never reaches it.

The effect is measurable, and it is the reason for the change: **11 of the 22 end events, and 26 of the 38 user tasks, are now reachable from the start event without passing a single message wait.** The pathway can be started, worked and finished entirely in Tasklist, with the external workers doing every service task.

The patient pool keeps its hand-offs. They are now the hospital telling the patient — `OUT_Auto_DispatchAppointmentLetter` and `OUT_Auto_SendFollowUpLetter` each carry an appointment notification to `P_Patient` — rather than the patient telling the engine. The message-flow count is unchanged at 15.

The convergences are unmerged, and that was checked rather than assumed. Twenty-one activities in this model have more than one incoming sequence flow with no merging gateway between them. `tools/analyse_bpmn.py` has always counted them; what the count does not say is whether they are *safe*, and the two are not the same question. An uncontrolled merge is only a defect when two of its incoming flows can carry a token at the same time — then the activity fires once per token, and for a service task that sends a letter that means a duplicate letter.

`tools/audit_convergences.py` answers that question and fails the build if the answer is ever yes. On this model: **17 of the 21 have a loop-back among their incoming flows**, so the second arrival can only come round after the first has finished (the retry paths — the letter that was rejected, the funding that needed chasing, the cycle that was delayed);

the other **4 come from a single exclusive fork or a linear stretch**, so only one token exists. **None can double-fire, and none needs a merging gateway.** The tool was checked against two deliberately broken variants — one where both branches of the `OUT_GW_NotifyChannels` fork land on `OUT_Auto_DispatchAppointmentLetter`, and one where the second branch reaches it round an intermediate task — and it fails both.

That is a different position from the earlier "20 unmerged convergences" line, which recorded the count and left the reader to wonder. The count is the same finding; the safety is now demonstrated rather than asserted, and the demonstration is executable.

One date is deliberately left unconstrained. `record-attendance.attendanceDate` is the day the patient came in. Every other record date in the model is held to "not in the future" by the worker-side `DateRuleValidator` (`referralDate`, `decisionDate`, `consultationDate`, …), and this one was at first. It was removed on 2026-10-16, because it fights the way the pathway has to be demonstrated: the appointment is booked for a date next week and the attendance is then recorded immediately, so **the natural answer — the appointment date — is in the future**, and the rule turned an ordinary demonstration into an incident on `CON_Auto_AuthoriseRequest` with three retries burned. A validation rule that fails on the normal path is worse than no rule; the field is therefore listed under `freeDates` in `bpm/workers/src/main/resources/date-rules.json` with that reason, and `tools/harden_forms.py` reports it as `deliberately-unconstrained` so the decision is visible rather than looking like an oversight.

One pool, but six outside participants. The Task asks for more than one pool "where the case requires collaboration between independent participants". The referring organisation, the patient and the four outside services are independent participants, so they stay as pools and the exchanges with them stay as message flows. What is *not* a participant is the hospital's own finance desk.

External services are service tasks, not message flows to a system pool. The scheduling service, the correspondence service, the payment provider and the capacity provider are systems the hospital calls. Each exchange is drawn once, at the activity that establishes it: the request leaves the hospital-side service task and the reply arrives on it. The alternative — a send task and a receive task for every call — would add eight elements and duplicate activities the workers already perform. The simulation is stated as a limitation in `14-feedback-response.md`.

The supplier row sits above the pool, not below it. Twelve of the fifteen messages start or end in the top four lanes. Under the pool every one of those lines would run the full height of a 3,550 px drawing; above it, only the three payment and refund lines do. `tools/bpmn_builder_v3.py` routes them on their own tracks through a dedicated band.

Twenty-one activities receive more than one sequence flow without a merging gateway. This is valid BPMN and Camunda 8 executes the convergence as a pass-through, and the incoming flows are alternatives — the difference between "the charge failed" and "the charge was taken" arriving at the same recording step. Drawing a gateway for each would add twenty-one columns to a diagram that is already 74 columns wide and would change no behaviour.

The v14.0 release justified this partly by the deployment batch ceiling: extra gateway elements grow the resource stored in every process-definition record. v16.0 is one process in a 202 KB file, so that reason has gone, and the honest position is narrower. The engine handles the merge correctly; what does not hold up is the reading. "Two arrows arrive and nothing says they are alternatives" is a fair criticism of the drawing, and pointing at FEEL's pass-through semantics does not answer it. It is counted rather than hidden — `tools/analyse_bpmn.py` reports it as `unmergedConvergingTasks`, currently 20 — and it is the largest known weakness in the diagram.

Lanes are nested, not flat. The Task asks the model to represent responsibilities. A flat list of nine team lanes does not say who answers for what; three responsibility lanes holding the teams that answer for them does, and it is the same split the strategic model uses. The BPMN is written as a `childLaneSet`, which is what the schema and the Modeler expect.

---

5. It was deployed and run

The rebuild was not signed off on a parse. It was deployed to a local Camunda 8 (c8run 8.10.0-alpha5, JDK 21, with `bpm/workers/target/hospital-external-workers-1.0.0.jar` running the 36 job workers) and driven end to end by `tools/run_v16.py`, which completes each user task the way Tasklist does and feeds each message catch event the message it subscribes to.

Scenario	Driver scenario	Result
The normal pathway	`--scenario normal`	**COMPLETED**, 26 steps
Patient declines at the consultation	`--scenario decline`	**COMPLETED**, 14 steps, ends at *Patient does not proceed with treatment*
Referral pack fails the checklist	`--scenario incomplete-pack`	**COMPLETED**, 31 steps: round the missing-information loop, pack re-supplied, pathway continues
No appointment slot in the window	`--scenario no-slots`	**COMPLETED**, 10 steps: escalate to the Pathway Coordinator, who writes to the patient
Appointment over, refund due	`--scenario refund`	**COMPLETED**, 28 steps through the refund decision
Appointment over, no refund due	`--scenario no-refund`	**COMPLETED**, 27 steps
An enquiry rather than a referral	`--scenario enquiry`	**COMPLETED**, 4 steps, through the message start event

```
scenario: normal
  created from the none start event  (correlationKey=R6-normal)
  instance 2251799814715785
   1  user task   SEC_Task_CheckPack           Check the referral pack against the document checklist
   3  user task   CON_Task_ReviewReferral      Review the referral and record the clinical decision
   5  user task   SEC_Task_NotifyReferrer      Record the decision and notify the referring organisation
   7  user task   OUT_Task_ChooseSlot          Choose and confirm the appointment
   9  message in  OUT_Catch_PatientAttended    appointment.attended
  11  user task   CON_Task_RecordConsent       Record consent and complete the treatment booking request
  13  user task   CON_Task_DraftLetter         Draft and approve the clinic letter
  15  user task   SEC_Task_ProcessLetter       Check and process the approved clinic letter
  17  user task   FIN_Task_DetermineFunding    Establish the funding route and record the authorisation
  19  user task   CON_Task_ReviewCycle         Review the blood results and decide on the next cycle
  21  user task   OUT_Task_ConfirmFollowUp     Confirm the follow-up appointment
  23  message in  OUT_Catch_AppointmentOutcome appointment.outcome
  25  user task   OUT_Task_DecideNextAction    Decide whether another appointment is offered

instance state: COMPLETED after 26 steps
```

Logs: `screenshots/run-v16-*.log`. Engine-side record — the deployments, the instance states, the incidents and the element-by-element history of a completed instance: `screenshots/v16/engine-record.txt`.

Four defects the run found, which reading the model had not

Each of these stopped the instance at the element named. They are the reason this section exists.

1. **`SEC_Auto_ValidateDocuments` — `UNHANDLED_ERROR_EVENT`.** The worker raises *two* error codes from that one activity: `REFERRAL_PACK_INCOMPLETE` and `REFERRAL_PACK_UNREADABLE`. The model had a boundary event for the first only, so an unreadable pack stopped the instance outright. Fixed by adding `SEC_Bnd_PackUnreadable`, routed to the same *ask the referring organisation* step.
2. **`SEC_Throw_InfoRequested` — `JOB_NO_RETRIES`.** Every message throw event needs a `zeebe:header` named `messageName` for the `publish-message` worker. The draft had the task definition but not the header. Fixed on all four throw events.
3. **`FIN_Auto_PreparePayment` — `JOB_NO_RETRIES`.** The run supplied no `treatmentPlan`, so the charge worker raised `CHARGE_CALCULATION_FAILED`; the model caught that and routed to manual pricing, which then handed on a blank `chargeAmount`. This one was the **driver** supplying too little, not the model — the model's behaviour was right, and it is recorded here because a defect log that only lists model faults is not a defect log.
4. **`TRT_GW_MoreCycles` — `EXTRACT_VALUE_ERROR`.** The gateway tested `cycleNumber < cycleCount`, but the scheduling worker returns `nextCycleNumber` and nothing anywhere set `cycleNumber`, so the condition evaluated `null < 6`. Fixed with an output mapping on `TRT_Auto_ScheduleCycle`: `=nextCycleNumber -> cycleNumber`.

Reconciling the model against the workers' declared variables would not have found any of these. Running it found all four.

The second round: what a systematic check found that the run had not

The first round of defects was found by running. The second was found by asking the model and the forms the same question in every place, which is what four checks in `tools/spec_v3.py` now do at build time. All four refuse to generate the model.

Check	The question it asks
`check_conditions`	does a condition compare a form field against the right type?
`check_gateway_variables`	is the variable it tests written by **anything**?
`check_gateway_sources`	is it written by something **upstream** of the gateway?
`check_option_values`	does the model test a value the form actually offers?

They found eight more defects of one family, and the run had passed over every one of them because the driver was posting its own variable names rather than the fields the forms contain.

1. **`consentGiven` compared against a boolean.** The form submits the string `"true"`; the model tested `= consentGiven = true`. FEEL does not consider those equal, so **every patient who consented was routed to the decline branch**. Nine of the ten form-backed booleans were written correctly and this one was not — precisely the defect the earlier release had claimed to fix as DEF-02.
2. **`modificationAuthorised` — nothing set it.** The gateway deciding whether a treatment change applies read a variable no form field and no worker wrote, so it took its default branch every time: every authorised change was refused. The form now asks the question the gateway reads.

3. `outcomeAction` **— nothing set it.** The same shape again, on "what happens after the appointment". The model now uses `cancellationAction`, which the form offers.
4. `requiresPhoneCall` **— nothing set it.** The inclusive gateway that decides whether to telephone the patient could never take the telephone branch.
5. `answerableNow` **was answered by a later form than the gateway that reads it.** The call handler decides whether an enquiry can be answered on the call; the field lived on a form completed after that decision.
6. `delayAction` **tested a value the form does not offer.** The model tested `"REPRIORITISE"`; the form offers `WIDEN_SEARCH`.
7. `appointmentOutcome` **offered values the recording worker rejects.** The form offered `CANCELLED_BY_PATIENT` and `CANCELLED_BY_HOSPITAL`; the worker accepts `ATTENDED / CANCELLED / DECLINED / DNA / DID_NOT_ATTEND` and fails the job otherwise. `paymentStatus` had the same problem with `INCOMPLETE`, and `declinedAction` offered a branch the model never tested while testing one (`SEEK_AUTHORISATION`) the form never offered — so the urgent-treatment authorisation was unreachable.
8. `cycleNumber` **was reset by the form that displays it.** The review form made the reviewer retype the cycle counter, and the counter is what the "are there further cycles" gateway tests. The pathway cycled for ever. The field is now read-only, which is what it should always have been.

And one defect that was the new tooling's, not the project's: the run driver's own form filler put the *label* of a text field in as its value, so `patientRef` became the literal string `Patient reference` and every message subscription waited on the correlation key `Patient reference-referral-info`. The driver now submits only the fields a form requires, in the types that form submits, with the case data taking precedence over the form's defaults.

Compensation

v14.0 modelled compensation. v16.0's first draft did not — and the documents still claimed a live compensation run that the cited log does not contain, because there is no `compensat` anywhere in it. That claim is now marked as wrong where it stands, and the construct is back in the model where the business genuinely needs it: committing the treatment series is the point of no return, so `TRT_Auto_ConfirmAppointments` carries a compensation boundary event, `TRT_Comp_ReleaseSeries` is its handler (`isForCompensation="true"`, no incoming flow), and `TRT_Throw_Compensate` throws it when the plan changes mid-course. `screenshots/run-v16-compensation.log` is the scenario that runs it, and it completes.

6. What is still not proved

- The five outside participants are black-box pools. Their internal behaviour is not modelled; the hospital-side workers simulate it.
- Twenty implicit convergences remain (§4) — `tools/analyse_bpmn.py` reports `unmergedConvergingTasks: 20`. They execute correctly — the runs above pass through several — but they are still drawn without merging gateways.
- The boundary timers (`P14D`, `P7D`, `P2D`, `PT1H`) and the weekly sweep trigger (`R/PT168H`) were not exercised. Reaching them means waiting or advancing the engine clock, and no timer fired during these runs.
- The supplier-failure paths that depend on the workers' own simulation flags (`DEMO_PAYMENT_STATUS=DECLINED`, `DEMO_EXTERNAL_UNAVAILABLE_RESOURCES`, `DEMO_PAYMENT_CONFIRMATION_LOST_MODE`) were not run in this pass, except `DEMO_SCHEDULING_FORCE_NO_SLOTS`, which the no-slot scenario needs.

- The demonstration is a driven run, not a person using Tasklist. Every step is the same API call Tasklist makes, but nobody clicked a form, and the evidence is a log rather than a screen recording.